



DOCUMENTO DE ESPECIFICAÇÃO TÉCNICA

Finance Pro

Aplicativo multiplataforma de monitoramento de ativos financeiros — arquitetura, decisões técnicas e contratos de integração.

PROJETO

easy_finance

VERSÃO

1.0.0+1

PLATAFORMA

Flutter 3.41 / Dart 3.11

DATA

13/08/2026

— Sumário

1	Visão Geral do Produto	10	Autenticação e Segurança
2	Stack Tecnológica	11	Contrato da API
3	Arquitetura do Sistema	12	Persistência Local
4	Estrutura de Diretórios	13	Design System e Temas
5	Camada de Domínio	14	Internacionalização
6	Camada de Dados	15	Estratégia de Testes
7	Camada de Apresentação	16	Registro de Decisões Técnicas
8	Gerenciamento de Estado	17	Build, Configuração e Distribuição
9	Navegação e Roteamento	18	Limitações Conhecidas e Roadmap

Escopo deste documento

Este documento descreve a arquitetura, as decisões técnicas e os contratos de integração da aplicação cliente **Finance Pro**. Ele cobre exclusivamente o aplicativo Flutter; o serviço de backend HTTP consumido pela aplicação é tratado como sistema externo, documentado aqui apenas pela ótica do contrato consumido (seção 11).

Público-alvo

Engenheiros de software responsáveis pela manutenção e evolução do cliente, revisores técnicos e responsáveis pela integração com o backend.

Métricas do código-fonte



Contagem de linhas exclui código gerado (`*.freezed.dart` , `*.g.dart` , `app_localizations*.dart`), que soma volume adicional relevante mas não é mantido manualmente.

01 Visão Geral do Produto

Finance Pro é um aplicativo de acompanhamento de carteira de observação (*watchlist*) de ativos financeiros — ações, ETFs e criptomoedas — com cotações, séries históricas, notícias agregadas e um assistente conversacional.

1.1 Funcionalidades

MÓDULO	DESCRIÇÃO FUNCIONAL	SITUAÇÃO
Autenticação	Cadastro de usuário com perfil de investidor, login por e-mail/senha via OAuth2 <i>password grant</i> , sessão persistida e logout com limpeza de credenciais.	INTEGRADO
Início (Home)	Watchlist com cotação e direção de variação por ativo, reordenação por arrastar-e-soltar, gráficos históricos com seletor de período (1D/1W/1M/1Y/ALL), carrossel automático de notícias e <i>pull-to-refresh</i> .	INTEGRADO
Perfil	Exibição e edição dos dados do usuário, busca de ativos com <i>debounce</i> , seleção por chips e sincronização diferencial da watchlist.	INTEGRADO
Assistente (IA Chat)	Interface conversacional com renderização Markdown, indicador de digitação e respostas contextuais sobre conceitos de investimento.	MOCK LOCAL
Configurações	Alternância de tema claro/escuro, seleção de idioma (PT-BR / EN-US), informações de versão e encerramento de sessão com confirmação.	INTEGRADO

1.2 Plataformas suportadas

O projeto mantém *runners* nativos para **Android** (com `example.easy_finance`), **iOS** (deployment target 13.0), **Web**, **Windows**, **macOS** e **Linux**. O foco de produto e de validação de UX é mobile; os demais alvos são funcionais por consequência do *toolkit*, e não receberam ajuste específico de layout responsivo.

1.3 Premissas do domínio

- Todos os preços são expressos e formatados em **dólar norte-americano** (`price_usd` , `NumberFormat.simpleCurrency(locale: 'en_US')`).
- A direção do movimento do ativo é fornecida pelo backend como *string* de domínio em português (`"subindo"` / demais valores) e não é derivada no cliente a partir da série histórica.
- Tickers seguem a nomenclatura do provedor de mercado a montante (ex.: `AAPL` , `BTC-USD`), sem normalização no cliente.

02 Stack Tecnológica

2.1 Plataforma base

COMPONENTE	VERSÃO	PAPEL
Flutter SDK	3.41.1 (stable)	Framework de UI e camada de renderização multiplataforma.
Dart SDK	^3.11.0	Linguagem; restrição declarada em <code>pubspec.yaml</code> .
Material 3	<code>useMaterial3: true</code>	Sistema de componentes base sobre o qual o tema próprio é construído.

2.2 Dependências de produção

PACOTE	VERSÃO	JUSTIFICATIVA DE ADOÇÃO
<code>flutter_riverpod</code>	<code>^2.5.1</code>	Injeção de dependência e gerência de estado reativa com escopo compilado e testabilidade por sobrescrita de provider.
<code>riverpod_annotation</code>	<code>^2.6.1</code>	Anotações para geração de providers; disponível para adoção incremental.
<code>dio</code>	<code>^5.7.0</code>	Cliente HTTP com suporte a interceptadores, essencial para injeção de token e tratamento centralizado de 401.
<code>flutter_secure_storage</code>	<code>^9.2.2</code>	Armazenamento de credenciais no Keychain (iOS) / EncryptedSharedPreferences (Android).
<code>go_router</code>	<code>^14.6.0</code>	Roteamento declarativo com suporte nativo a <i>redirect</i> assíncrono e shell de navegação com estado preservado.
<code>fl_chart</code>	<code>^0.69.0</code>	Renderização dos gráficos de linha das séries históricas.
<code>cached_network_image</code>	<code>^3.4.1</code>	Cache em disco dos ícones de ativos, reduzindo requisições repetidas.
<code>shimmer</code>	<code>^3.0.0</code>	Estados de carregamento por <i>skeleton</i> , evitando deslocamento de layout.
<code>intl</code>	<code>^0.20.2</code>	Formatação de moeda e datas sensível a <i>locale</i> .
<code>shared_preferences</code>	<code>^2.3.3</code>	Persistência de preferências não sensíveis (tema, idioma, ordem da watchlist).
<code>flutter_markdown</code>	<code>^0.7.4</code>	Renderização das respostas do assistente, que utilizam Markdown (negrito, listas, tabelas).
<code>freezed_annotation</code>	<code>^2.4.4</code>	Contrato para modelos imutáveis com <code>copyWith</code> e igualdade estrutural.
<code>json_annotation</code>	<code>^4.9.0</code>	Contrato de serialização JSON.

2.3 Dependências de desenvolvimento

PACOTE	VERSÃO	PAPEL
build_runner	^2.4.13	Orquestrador da geração de código.
freezed	^2.5.7	Geração das <i>data classes</i> imutáveis.
json_serializable	^6.9.0	Geração de <code>fromJson</code> / <code>toJson</code> .
riverpod_generator	^2.6.3	Geração de providers anotados.
mocktail	^1.0.4	Dublês de teste sem geração de código, permitindo <i>mocks</i> de interfaces do domínio.
flutter_lints	^6.0.0	Conjunto de regras de análise estática.

2.4 Configuração do analisador

O arquivo `analysis_options.yaml` estende `package:flutter_lints/flutter.yaml` e suprime duas regras cujos disparos são falsos positivos originados por geração de código:

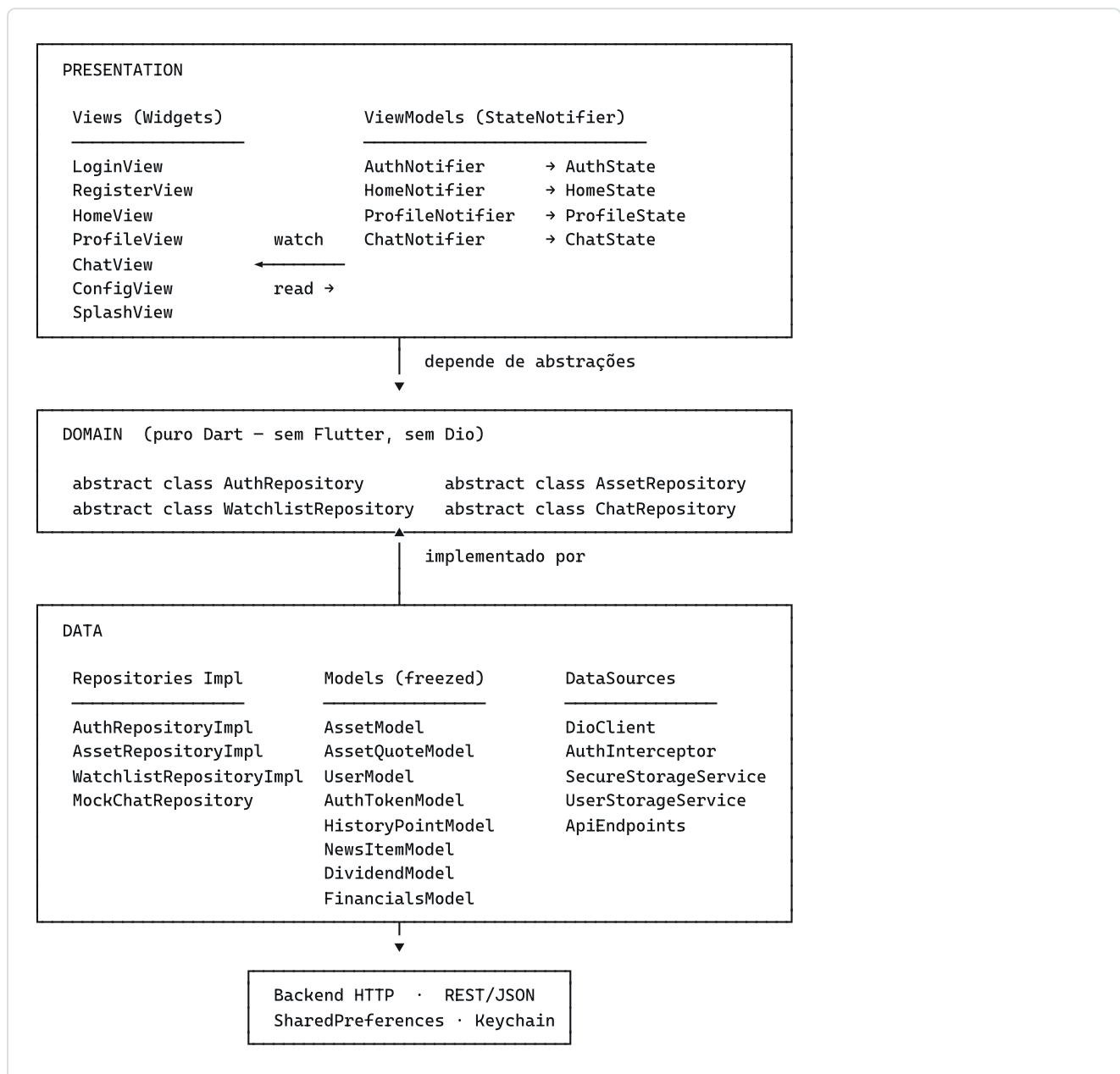
```
analyzer:
  errors:
    invalid_annotation_target: ignore # @JsonKey em parâmetros de factory (freezed 2.x)
    unnecessary_underscores: ignore  # gerado por code gen
```

A supressão é declarada no nível do projeto, e não por `// ignore:` espalhado nos arquivos, mantendo os arquivos-fonte livres de ruído. Essa escolha tem o custo de também silenciar eventuais ocorrências legítimas em código autoral.

03 Arquitetura do Sistema

O sistema adota **Clean Architecture em três camadas** combinada ao padrão **MVVM** na camada de apresentação, com inversão de dependência mediada por interfaces de repositório e resolução via Riverpod.

3.1 Diagrama de camadas



3.2 Regra de dependência

As dependências apontam sempre para dentro. A camada de apresentação conhece apenas as abstrações declaradas em `lib/domain/repositories/`; nenhuma *view* ou *view model* importa `package:dio`. A resolução da implementação concreta ocorre em tempo de execução, através dos providers declarados junto de cada implementação:

```
// lib/data/repositories/asset_repository_impl.dart
final assetRepositoryProvider = Provider<AssetRepository>((ref) =>
  AssetRepositoryImpl(ref.read(dioClientProvider)));
```

O tipo exposto pelo provider é a **abstração** (`Provider<AssetRepository>`), não a implementação. Isso permite substituir a implementação em testes por `overrideWithValue` sem qualquer alteração no código de produção.

Desvio consciente do modelo canônico

Os modelos de dados residem em `lib/data/models/` e são referenciados diretamente pelas interfaces do domínio, em vez de existirem entidades de domínio separadas com mapeadores. A duplicação `model↔entity` foi considerada desproporcional para um cliente cujo modelo de leitura espelha o contrato da API. O custo dessa escolha é que uma mudança de *schema* no backend propaga até a camada de apresentação sem barreira intermediária.

3.3 Fluxo de uma requisição

Exemplo completo — carregamento da tela inicial:

```
HomeView.build()
├── ref.watch(homeNotifierProvider) → reconstrói ao mudar HomeState
├── HomeNotifier() [construtor] → load()
│   ├── watchlistRepo.getWatchlist() → GET /profile/watchlist
│   ├── _applyOrder(...) → SharedPreferences['watchlist_order']
│   ├── Future.wait( getQuote(t) vt ) → GET /assets/{ticker} [paralelo]
│   ├── Future.wait( getHistory(t) vt ) → GET /assets/{ticker}/history [paralelo]
│   ├── state = state.copyWith(quotes, history, loading: false)
│   └── _loadNews(tickers) [não aguardado]
│       ├── Future.wait( getNews(t) vt ) → GET /assets/{ticker}/news
│       └── state = state.copyWith(news, newsLoading: false)
```

Duas características do fluxo merecem destaque. Primeiro, as chamadas por ticker são disparadas em paralelo via `Future.wait`, de modo que a latência total tende ao maior tempo individual, e não à soma. Segundo, o carregamento de notícias é **deliberadamente não aguardado**: a tela renderiza cotações e gráficos assim que disponíveis, e a faixa de notícias preenche-se em seguida, com *skeleton* próprio.

04 Estrutura de Diretórios

lib/	
main.dart	Bootstrap: ensureInitialized + ProviderScope
app.dart	MaterialApp.router; liga tema, locale e router
core/	Infraestrutura transversal, sem regra de negócio
constants/	
api_constants.dart	baseUrl (dart-define) + mapa de períodos UI→API
app_colors.dart	Paleta única do design system
router/	
app_router.dart	GoRouter, guarda de rota e shell de 4 abas
theme/	
app_theme.dart	ThemeData claro e escuro
theme_service.dart	ThemeNotifier + persistência da preferência
locale_service.dart	LocaleNotifier + persistência do idioma
utils/	
formatters.dart	Moeda, data, tempo relativo, rótulo de direção
validators.dart	Validação de e-mail, senha e campo obrigatório
widgets/	
empty_state.dart	Estado vazio reutilizável com CTA opcional
error_view.dart	Estado de erro com ação de nova tentativa
shimmer_loading.dart	ShimmerBox e AssetCardShimmer
domain/	
repositories/	Somente contratos abstratos (4 interfaces)
data/	
datasources/	
api_endpoints.dart	Catálogo centralizado de rotas
dio_client.dart	Provider do Dio configurado
auth_interceptor.dart	Injeção de Bearer + tratamento de 401
secure_storage_service.dart	Token JWT
user_storage_service.dart	Dados do usuário
models/	8 modelos freezed (+ .freezed.dart e .g.dart)
repositories/	4 implementações concretas
presentation/	
viewmodels/	4 StateNotifiers com estado imutável
views/	Telas, agrupadas por feature, com widgets/ locais
l10n/	
intl_en.arb / intl_pt.arb	Fontes de tradução (38 chaves)
app_localizations*.dart	Gerado por flutter gen-l10n

4.1 Convenções estruturais

- **Agrupamento por feature dentro da camada.** Cada tela possui um diretório próprio sob `views/`, com um subdiretório `widgets/` para componentes que só ela utiliza. Componentes usados por mais de uma feature sobem para `core/widgets/`.
- **Provider colocado junto do que ele constrói.** Não há arquivo central de *injection*; cada provider é declarado no final do arquivo do serviço ou repositório correspondente. Isso mantém a definição próxima do consumidor e evita um ponto único de acoplamento.

- **Espelhamento na árvore de testes.** `test/` reproduz a hierarquia de `lib/`, tornando previsível a localização do teste de qualquer unidade.

05 Camada de Domínio

A camada de domínio contém exclusivamente contratos. São quatro interfaces abstratas, sem dependência de Flutter, Dio ou qualquer detalhe de infraestrutura.

5.1 Contratos

```
abstract class AuthRepository {
  Future<AuthTokenModel> login(String email, String password);
  Future<UserModel> register({
    required String name,      required String email,
    required String password,  required String birthDate,
    required String investorProfile,
  });
}

abstract class AssetRepository {
  Future<List<AssetModel>> search(String query);
  Future<AssetQuoteModel> getQuote(String ticker);
  Future<List<HistoryPointModel>> getHistory(String ticker, String period);
  Future<List<NewsItemModel>> getNews(String ticker);
  Future<List<DividendModel>> getDividends(String ticker);
  Future<FinancialsModel> getFinancials(String ticker);
}

abstract class WatchlistRepository {
  Future<List<AssetModel>> getWatchlist();
  Future<void> addAssets(List<AssetModel> assets);
  Future<void> removeAssets(List<String> tickers);
}

abstract class ChatRepository {
  Future<String> sendMessage(String userMessage);
}
```

5.2 Propriedades de projeto

PROPRIEDADE	CONSEQUÊNCIA PRÁTICA
Ausência de camada de <i>use case</i>	A orquestração reside nos <i>view models</i> . Para o volume atual de regras — combinação de chamadas e transformação de listas — uma classe por caso de uso adicionaria indireção sem isolar comportamento novo. Se surgirem regras compartilhadas entre telas, esse é o ponto natural de extração.
Erros propagados como exceções	Não há tipo <code>Result</code> / <code>Either</code> . As implementações deixam <code>DioException</code> propagar, e cada <i>view model</i> converte a exceção em estado de erro no seu <code>catch</code> . A abordagem é mais leve, ao custo de a assinatura do método não declarar as falhas possíveis.
Granularidade fina do <code>AssetRepository</code>	Métodos separados por recurso (<code>getQuote</code> , <code>getHistory</code> , <code>getNews</code> ...) permitem que o chamador componha exatamente as chamadas necessárias e paralelize à vontade — usado intensivamente em <code>HomeNotifier.load()</code> .
<code>ChatRepository</code> minimalista	Uma única operação <code>sendMessage</code> , sem histórico de conversa no contrato. A troca do dublê local por um serviço real de IA exige apenas uma nova implementação e a alteração de uma linha no provider.

Métodos não consumidos

`getDividends` e `getFinancials` estão declarados, implementados e cobertos pelo contrato, mas nenhuma tela os consome no estado atual. Correspondem a uma tela de detalhe de ativo prevista e ainda não construída. São dívida deliberada, não código morto accidental.

06 Camada de Dados

6.1 Modelos

Todos os oito modelos são gerados por **freezed** combinado a **json_serializable**, obtendo imutabilidade, `copyWith`, igualdade estrutural, `toString` legível e serialização — sem código manual sujeito a divergência.

MODELO	CAMPOS
AssetModel	ticker, name, iconUrl
AssetQuoteModel	ticker, name, iconUrl, priceUsd, direction
UserModel	id, name, email, birthDate, investorProfile, isActive
AuthTokenModel	accessToken, tokenType
HistoryPointModel	date, close
NewsItemModel	title, link, publisher, summary?, providerPublishTime
DividendModel	date, amount
FinancialsModel	marketCap?, peRatio?, dividendYield?, targetPrice?, recommendation?

MAPEAMENTO DE NOMENCLATURA

O backend expõe campos em `snake_case`; o código Dart usa `lowerCamelCase`. A tradução é declarada campo a campo com `@JsonKey`, em vez de uma estratégia global de conversão de nomes:

```
@freezed
class AssetQuoteModel with _$AssetQuoteModel {
  const factory AssetQuoteModel({
    required String ticker,
    required String name,
    @JsonKey(name: 'icon_url') required String iconUrl,
    @JsonKey(name: 'price_usd') required double priceUsd,
    required String direction,
  }) = _AssetQuoteModel;

  factory AssetQuoteModel.fromJson(Map<String, dynamic> json) =>
    _$AssetQuoteModelFromJson(json);
}
```

A anotação explícita é mais verbosa, porém torna o contrato auditável na leitura do próprio modelo e resiste a campos que fujam do padrão.

OBRIGATORIEDADE DOS CAMPOS

`FinancialsModel` declara **todos** os campos como anuláveis, refletindo que indicadores fundamentalistas são indisponíveis para diversas classes de ativo — criptomoedas não possuem P/E nem *dividend yield*. Os demais modelos usam `required`, o que faz a desserialização falhar de forma explícita e imediata diante de uma resposta incompleta, em vez de propagar `null` silenciosamente até a UI.

6.2 Fontes de dados

CLIENTE HTTP

```
final dioClientProvider = Provider<Dio>((ref) {  
  final dio = Dio(BaseOptions(  
    baseUrl: ApiConstants.baseUrl,  
    connectTimeout: const Duration(seconds: 15),  
    receiveTimeout: const Duration(seconds: 30),  
    headers: {'Content-Type': 'application/json'},  
  ));  
  final storage = ref.read(secureStorageServiceProvider);  
  final onLogout = ref.read(logoutCallbackProvider);  
  dio.interceptors.add(AuthInterceptor(storage, onLogout: onLogout));  
  return dio;  
});
```

Os *timeouts* são assimétricos por intenção: 15 s para estabelecer conexão e 30 s para receber a resposta, acomodando *endpoints* de série histórica com período `max`, cuja carga útil é substancialmente maior.

CATÁLOGO DE ROTAS

`ApiEndpoints` centraliza todos os caminhos como constantes ou funções puras. Nenhuma URL aparece literal em repositório ou *view model*, de modo que uma alteração de rota no backend tem ponto único de manutenção — e é diretamente testável, como comprovam os 13 casos de `api_endpoints_test.dart`.

6.3 Implementações de repositório

As implementações são deliberadamente finas: montam a requisição, desserializam a resposta e retornam. Não realizam cache, retry, log nem tratamento de erro — responsabilidades atribuídas ao interceptador (autenticação) e aos *view models* (apresentação do erro).

```
@override  
Future<List<HistoryPointModel>> getHistory(String ticker, String period) async {  
  final r = await _dio.get(  
    ApiEndpoints.assetHistory(ticker),  
    queryParameters: {'period': period},  
  );  
  return (r.data as List)  
    .map((e) => HistoryPointModel.fromJson(e as Map<String, dynamic>))  
    .toList();  
}
```

SINCRONIZAÇÃO DIFERENCIAL DA WATCHLIST

`WatchlistRepository` expõe operações em lote (`addAssets`, `removeAssets`) em vez de operações unitárias. O `ProfileNotifier` mantém a lista original e a lista selecionada em memória, calcula a diferença no momento de salvar e emite no máximo duas requisições — evitando uma chamada de rede por toque do usuário:

```

final toAdd = state.selectedAssets
    .where((a) => !originalTickers.contains(a.ticker)).toList();
final toRemove = state.originalWatchlist
    .where((a) => !selectedTickers.contains(a.ticker))
    .map((a) => a.ticker).toList();

if (toAdd.isNotEmpty) await watchlistRepo.addAssets(toAdd);
if (toRemove.isNotEmpty) await watchlistRepo.removeAssets(toRemove);

```

REPOSITÓRIO DE CHAT

`MockChatRepository` implementa o contrato com um dicionário de palavras-chave (`preço`, `tendência`, `dividendo`, `crypto`, `etf`, `perfil`, `watchlist`, `ação`, `recomendação`) mapeadas a respostas em Markdown, com variantes sorteadas e latência artificial entre 800 ms e 1.500 ms para simular processamento. Existe uma resposta genérica para entradas não reconhecidas. O objetivo é permitir o desenvolvimento e a validação completa da UI de conversa antes da disponibilidade do serviço de IA.

07 Camada de Apresentação

A apresentação segue MVVM: cada tela relevante possui um *view model* (`StateNotifier`) que expõe um estado imutável, e as *views* são funções desse estado.

7.1 Padrão de estado

Todo *view model* expõe uma única classe de estado imutável, com valores-padrão no construtor e `copyWith`. Estados de carregamento, erro e dados coexistem no mesmo objeto, o que elimina combinações inconsistentes entre variáveis independentes:

```
class HomeState {
  final bool loading;
  final bool refreshing;
  final String? error;
  final List<AssetQuoteModel> quotes;
  final String selectedPeriod;
  final Map<String, List<HistoryPointModel>> history;
  final List<NewsItemModel> news;
  final bool newsLoading;

  const HomeState({ this.loading = true, this.refreshing = false, ... });
  HomeState copyWith({ ... });
}
```

Distinção entre loading e refreshing

A primeira carga exibe *skeleton* em tela cheia; o *pull-to-refresh* mantém o conteúdo atual visível e mostra apenas um indicador discreto no cabeçalho. Sem os dois sinalizadores separados, atualizar a tela faria o conteúdo desaparecer e reaparecer — regressão perceptível de qualidade.

Nota de implementação: em todos os `copyWith` do projeto, o campo `error` é atribuído diretamente (`error: error`) e não com o operador `??`. É intencional: permite **limpar** o erro em uma transição de estado bem-sucedida. A contrapartida é que qualquer `copyWith` que não repasse o erro o descarta — comportamento que precisa ser conhecido por quem edita esses métodos.

7.2 Inventário de view models

VIEW MODEL	ESTADO	RESPONSABILIDADES
AuthNotifier	AuthState {status, error}	Login, cadastro com auto-login subsequente, logout e tradução de erros HTTP para mensagens de usuário.
HomeNotifier	HomeState (8 campos)	Carga inicial e atualização, agregação paralela de cotações e históricos, troca de período, reordenação persistida e carga assíncrona de notícias.
ProfileNotifier	ProfileState (10 campos)	Leitura do usuário, busca de ativos com <i>debounce</i> , seleção/remoção local, sincronização diferencial e atualização de perfil.
ChatNotifier	ChatState {messages, typing}	Histórico de mensagens, mensagem de boas-vindas, indicador de digitação e degradação para mensagem de erro na falha.

7.3 Decisões de UX com impacto técnico

COMPORTAMENTO	IMPLEMENTAÇÃO
Busca com <i>debounce</i>	<code>ProfileNotifier.search</code> mantém um <code>Timer</code> de 400 ms cancelado a cada tecla; a requisição só parte na pausa da digitação. O timer é cancelado em <code>dispose</code> , prevenindo emissão de estado após o descarte.
Ordem manual da watchlist	<code>ReorderableListView</code> na Home; a nova ordem de tickers é gravada em <code>SharedPreferences</code> e reaplicada em <code>_applyOrder</code> a cada carga. Ativos ausentes da ordem salva são anexados ao final, o que mantém o comportamento correto quando a watchlist muda no servidor.
Notícias em carrossel	<code>NewsTicker</code> usa <code>PageController(viewportFraction: 0.65)</code> com <code>Timer.periodic</code> de 4 s e avanço circular. O timer é reiniciado em <code>didUpdateWidget</code> e cancelado em <code>dispose</code> .
Retorno tátil	<code>HapticFeedback</code> aplicado de forma graduada: <code>lightImpact</code> em navegação e ações comuns, <code>mediumImpact</code> ao iniciar o arraste e em ações destrutivas, <code>selectionClick</code> na troca de idioma.
Estados vazios acionáveis	<code>EmptyState</code> aceita rótulo e ação; a watchlist vazia não apenas informa, mas conduz o usuário à aba Perfil.
Confirmação de logout	Diálogo de confirmação obrigatório antes de encerrar a sessão, com verificação de <code>context.mounted</code> após o <code>await</code> — evita uso de contexto desmontado.

7.4 Componentes reutilizáveis

- `ShimmerBox` / `AssetCardShimmer` — *placeholders* com dimensões próximas ao conteúdo final, minimizando deslocamento de layout na troca.
- `ErrorView` — mensagem com ação opcional de nova tentativa, ligada diretamente ao método de recarga do *view model*.
- `EmptyState` — ícone, título, subtítulo e CTA opcional.
- `FloatingIAAvatar` — atalho flutuante para o assistente, com animação de pulso contínua (`repeat(reverse: true)`), escala 1,0→1,06).

08 Gerenciamento de Estado

Riverpod atua simultaneamente como contêiner de injeção de dependência e como mecanismo de estado reativo, eliminando a necessidade de uma biblioteca de DI separada.

8.1 Catálogo de providers

PROVIDER	TIPO	FUNÇÃO
<code>secureStorageServiceProvider</code>	<code>Provider</code>	Armazenamento seguro do token.
<code>userStorageServiceProvider</code>	<code>Provider</code>	Armazenamento seguro dos dados do usuário.
<code>dioClientProvider</code>	<code>Provider</code>	Cliente HTTP com interceptador acoplado.
<code>logoutCallbackProvider</code>	<code>StateProvider</code>	Ponto de extensão para reação global a 401.
<code>authRepositoryProvider</code>	<code>Provider<AuthRepository></code>	Resolve a implementação de autenticação.
<code>assetRepositoryProvider</code>	<code>Provider<AssetRepository></code>	Resolve a implementação de ativos.
<code>watchlistRepositoryProvider</code>	<code>Provider<WatchlistRepository></code>	Resolve a implementação de watchlist.
<code>chatRepositoryProvider</code>	<code>Provider<ChatRepository></code>	Resolve a implementação do assistente.
<code>sharedPreferencesProvider</code>	<code>FutureProvider</code>	Instância assíncrona de preferências.
<code>themeProvider</code>	<code>StateNotifierProvider</code>	<code>ThemeMode</code> corrente.
<code>localeProvider</code>	<code>StateNotifierProvider</code>	<code>Locale</code> corrente.
<code>appRouterProvider</code>	<code>Provider<GoRouter></code>	Configuração de rotas com acesso ao contêiner.
<code>authNotifierProvider</code>	<code>StateNotifierProvider</code>	Estado de autenticação.
<code>homeNotifierProvider</code>	<code>StateNotifierProvider</code>	Estado da tela inicial.
<code>profileNotifierProvider</code>	<code>StateNotifierProvider</code>	Estado da tela de perfil.
<code>chatNotifierProvider</code>	<code>StateNotifierProvider</code>	Estado da conversa.

8.2 Convenções de acesso

A distinção entre `ref.watch` e `ref.read` é aplicada de forma consistente:

- `ref.watch` — exclusivamente em `build`, para estabelecer a assinatura reativa que reconstrói o widget.
- `ref.read` — em *callbacks*, construtores e métodos, onde a reconstrução não é desejada.

Dentro dos *view models*, os repositórios são obtidos por `_ref.read(...)` no momento do uso, e não capturados no construtor. Isso permite que uma sobrescrita de provider feita em teste seja observada mesmo após a construção do *notifier*.

8.3 Ciclo de vida

`HomeNotifier` e `ProfileNotifier` disparam a carga inicial no próprio construtor. A consequência é que a primeira leitura do provider já inicia o tráfego de rede, sem necessidade de `initState` na view — e o estado é preservado enquanto o provider permanecer vivo, evitando recarga a cada troca de aba.

8.4 Testabilidade

A sobrescrita de providers é o mecanismo central de teste da camada de apresentação:

```
final container = ProviderContainer(  
  overrides: [  
    watchlistRepositoryProvider.overrideWithValue(mockWatchlist),  
    assetRepositoryProvider.overrideWithValue(mockAsset),  
  ],  
);  
addTearDown(container.dispose);
```

Nenhum *singleton* global, *service locator* ou estado estático precisa ser reinicializado entre testes, o que elimina uma classe inteira de falhas intermitentes por vazamento de estado.

Adoção parcial da API moderna

O projeto utiliza `StateNotifier`, e não a API mais recente `Notifier/AsyncNotifier`, ainda que `riverpod_generator` conste nas dependências de desenvolvimento. A escolha é coerente internamente — não há mistura de padrões — mas representa dívida técnica frente à direção do ecossistema Riverpod. A migração é mecânica e pode ser feita por *notifier*, sem alteração das views.

09 Navegação e Roteamento

9.1 Mapa de rotas

ROTA	TELA	ACESSO	OBSERVAÇÃO
/	SplashView	Pública	Verifica token e redireciona após ~1,2 s de animação.
/login	LoginView	Pública	Redireciona para <code>/home</code> se já houver sessão.
/register	RegisterView	Pública	Auto-login após cadastro bem-sucedido.
/home	HomeView	Protegida	Ramo 1 do shell de navegação.
/profile	ProfileView	Protegida	Ramo 2 do shell.
/chat	ChatView	Protegida	Ramo 3 do shell.
/config	ConfigView	Protegida	Ramo 4 do shell.

9.2 Shell com estado preservado

As quatro abas são declaradas como `StatefulShellRoute.indexedStack`, com um `StatefulShellBranch` por aba. Cada ramo mantém sua própria pilha de navegação e seu estado de rolagem, de modo que alternar entre abas não reconstrói a tela nem reposiciona a lista. A alternativa — um `IndexedStack` manual sob uma rota única — exigiria sincronizar índice e URL à mão.

```
StatefulShellRoute.indexedStack(
  builder: (context, state, shell) => AppShell(shell: shell),
  branches: [
    StatefulShellBranch(routes: [GoRoute(path: '/home', ...)]),
    StatefulShellBranch(routes: [GoRoute(path: '/profile', ...)]),
    StatefulShellBranch(routes: [GoRoute(path: '/chat', ...)]),
    StatefulShellBranch(routes: [GoRoute(path: '/config', ...)]),
  ],
)
```

Em `AppShell._onTap`, o parâmetro `initialLocation: index == shell.currentIndex` implementa o comportamento esperado de barra de abas: tocar na aba já ativa retorna à raiz daquele ramo, enquanto tocar em outra aba preserva onde o usuário estava.

9.3 Guarda de rota

```

redirect: (context, state) async {
    final storage = ref.read(secureStorageServiceProvider);
    final hasToken = await storage.hasToken();
    final loc = state.uri.toString();
    final publicRoutes = ['/', '/login', '/register'];

    if (!hasToken && !publicRoutes.contains(loc)) return '/login';
    if (hasToken && (loc == '/login' || loc == '/register')) return '/home';
    return null;
}

```

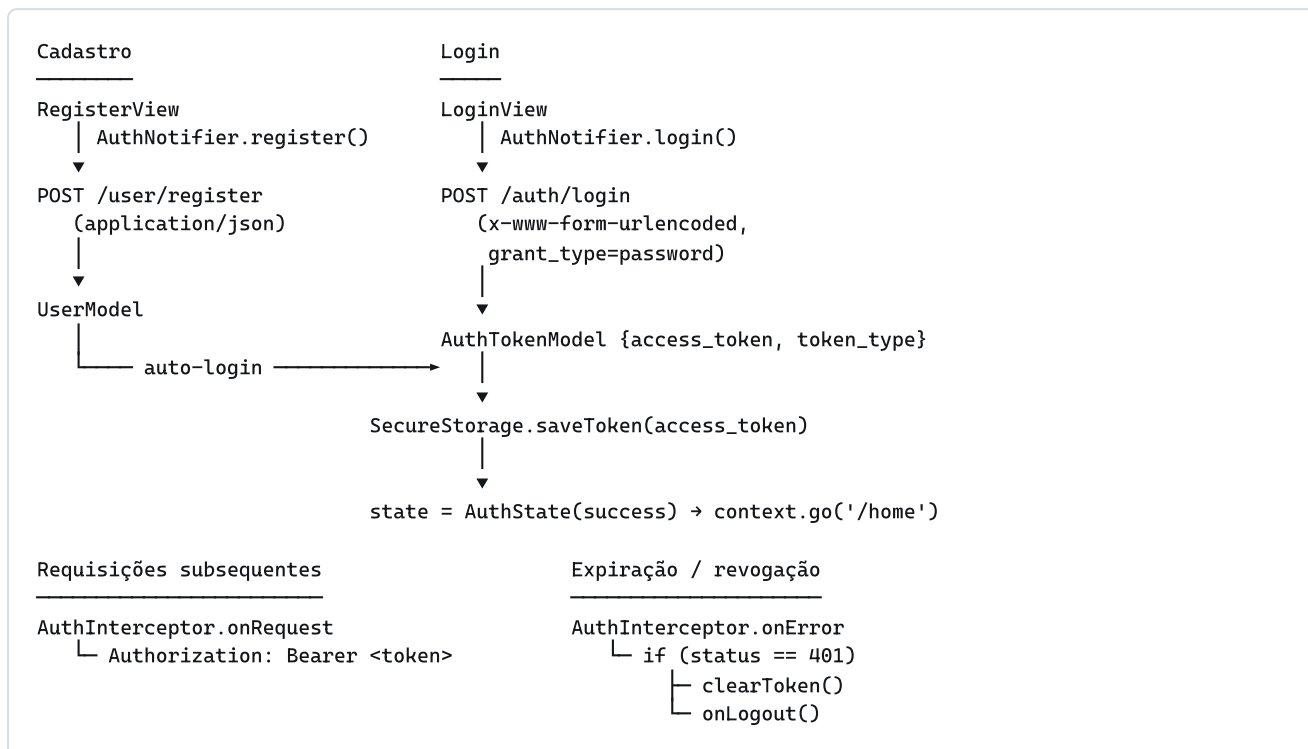
A guarda é **bidirecional**: além de bloquear rotas protegidas sem sessão, impede que um usuário autenticado retorne às telas de login e cadastro. Sendo assíncrona, consulta o armazenamento seguro a cada navegação, o que garante decisão sempre baseada no estado real de credencial — e não em cópia em memória potencialmente desatualizada por um 401 tratado pelo interceptador.

Custo e mitigação

Ler o Keychain/EncryptedSharedPreferences a cada navegação implica uma operação de I/O por transição. Para o volume de rotas deste aplicativo o impacto é imperceptível. Caso o número de rotas cresça, o caminho natural é manter um `StateProvider<bool>` de sessão em memória, sincronizado pelo interceptador e pelo fluxo de login, e usar `refreshListenable`.

10 Autenticação e Segurança

10.1 Fluxo de autenticação



10.2 Formato da requisição de login

O *endpoint* de login segue o padrão *OAuth2 Resource Owner Password Credentials*, exigindo `application/x-www-form-urlencoded` com os campos `username`, `password` e `grant_type` — convenção adotada por FastAPI com `OAuth2PasswordRequestForm`. O corpo é montado com `Uri.encodeComponent` em cada valor, prevenindo quebra do formato por caracteres reservados presentes em e-mails ou senhas:

```
data: 'username=${Uri.encodeComponent(email)}'
      '&password=${Uri.encodeComponent(password)}'
      '&grant_type=password',
options: Options(contentType: 'application/x-www-form-urlencoded'),
```

10.3 Armazenamento de credenciais

DADO	MECANISMO	JUSTIFICATIVA
Token de acesso	<code>flutter_secure_storage</code> chave <code>auth_token</code>	Credencial de portador; exige armazenamento cifrado pelo sistema operacional.
Dados do usuário	<code>flutter_secure_storage</code> chave <code>user_data</code>	Contém e-mail e data de nascimento — dado pessoal, tratado sob o mesmo nível de proteção do token.
Tema e idioma	<code>SharedPreferences</code>	Preferência de interface, sem valor para um atacante.
Ordem da watchlist	<code>SharedPreferences</code>	Estado de apresentação, reconstruível.

No Android, `flutter_secure_storage` é configurado com `AndroidOptions(encryptedSharedPreferences: true)`, o que direciona o armazenamento para `EncryptedSharedPreferences` com chave gerenciada pelo Android Keystore, em vez do arquivo de preferências em texto claro.

10.4 Tratamento centralizado de sessão expirada

`AuthInterceptor` concentra duas responsabilidades transversais, mantendo repositórios e *view models* livres de lógica de autenticação:

- `onRequest` — lê o token e, se presente, injeta o cabeçalho `Authorization: Bearer <token>`. Nenhum repositório manipula cabeçalhos de autenticação.
- `onError` — ao detectar HTTP 401, limpa o token armazenado e invoca `onLogout`. Como a guarda de rota consulta o armazenamento a cada navegação, a próxima transição já redireciona para `/login`.

10.5 Superfície de segurança e limitações

ASPECTO	SITUAÇÃO	NOTA
Token em armazenamento cifrado	SIM	Keychain (iOS) / Keystore + EncryptedSharedPreferences (Android).
Limpeza de credencial em 401	SIM	Centralizada no interceptador.
Validação de entrada no cliente	SIM	<code>Validators</code> : formato de e-mail, senha ≥ 6 caracteres, campos obrigatórios.
Mensagens de erro sem vazamento	SIM	<code>_extractError</code> traduz códigos HTTP para mensagens genéricas ao usuário.
<i>Refresh token</i>	AUSENTE	O backend retorna apenas <code>access_token</code> . Expirada a sessão, o usuário reautentica manualmente.
Certificate pinning	AUSENTE	Recomendado antes de distribuição em produção.
HTTPS obrigatório	CONFIGURÁVEL	O valor-padrão de <code>API_BASE</code> é <code>http://localhost:8000</code> , adequado a desenvolvimento. Builds de produção devem sobrescrever com origem HTTPS via <code>--dart-define</code> .
Ofuscação de binário	NÃO CONFIGURADA	<code>--obfuscate --split-debug-info</code> recomendado no pipeline de release.

Requisito para produção

A ausência de *certificate pinning* combinada à possibilidade de build apontar para origem HTTP torna o tráfego suscetível a interceptação em redes hostis. Antes de qualquer distribuição pública, é necessário: (1) fixar `API_BASE` em HTTPS no pipeline de build; (2) avaliar a adoção de pinning via `badCertificateCallback` do Dio; (3) habilitar ofuscação no release.

11 Contrato da API

Base URL definida em tempo de compilação por `String.fromEnvironment('API_BASE')`, com *fallback* `http://localhost:8000`. Todos os *endpoints* protegidos exigem o cabeçalho `Authorization: Bearer <token>`, injetado automaticamente.

11.1 Endpoints consumidos

MÉTODO	CAMINHO	PAYLOAD / QUERY	RESPOSTA
POST	/auth/login	form: username, password, grant_type	AuthTokenModel
POST	/user/register	json: name, email, password, birth_date, investor_profile	UserModel
PUT	/user/update/{id}	json: id, name, email, investor_profile	—
DELETE	/user/delete/{id}	—	—
GET	/assets	?search={query}	AssetModel[]
GET	/assets/{ticker}	—	AssetQuoteModel
GET	/assets/{ticker}/history	?period={1d 5d 1mo 1y max}	HistoryPointModel[]
GET	/assets/{ticker}/news	—	NewsItemModel[]
GET	/assets/{ticker}/dividends	—	DividendModel[]
GET	/assets/{ticker}/financials	—	FinancialsModel
GET	/profile/watchlist	—	AssetModel[]
POST	/profile/watchlist/add	json: [{ticker, name, icon_url}]	—
POST	/profile/watchlist/remove	json: [{ticker}]	—

DELETE `/user/delete/{id}` está declarado em `ApiEndpoints` e coberto por teste, mas ainda não é acionado por nenhuma tela — corresponde à funcionalidade de exclusão de conta prevista.

11.2 Tradução de períodos

Os rótulos de período exibidos ao usuário não coincidem com os valores aceitos pela API. A conversão é centralizada em `PeriodMap`, evitando que a nomenclatura do provedor de dados vaze para a interface:

RÓTULO NA UI	VALOR NA API	JANELA
1D	1d	Um dia
1W	5d	Cinco pregões (uma semana útil)
1M	1mo	Um mês — período padrão da aplicação
1Y	1y	Um ano
ALL	max	Série completa disponível

Toda conversão aplica *fallback* para `'1mo'` em caso de chave desconhecida, de modo que um rótulo inválido degrada para o comportamento padrão em vez de gerar exceção.

11.3 Tratamento de erros HTTP

CÓDIGO	TRATAMENTO	MENSAGEM AO USUÁRIO
401	Interceptor limpa o token e dispara <code>onLogout</code> ; no fluxo de login, é traduzido pelo <i>view model</i> .	"Email ou senha incorretos."
400	Tratado no cadastro.	"Email já cadastrado."
422	Falha de validação do backend.	"Dados inválidos."
Rede	<code>SocketException</code> / falha de conexão.	"Sem conexão com o servidor."
Demais	<i>Fallback</i> genérico.	"Erro inesperado. Tente novamente."

Falhas em *endpoints* não críticos são absorvidas sem interromper a tela. O carregamento de notícias, por exemplo, aplica `.catchError((_) => <NewsItemModel>[])` por ticker: um ativo sem notícias disponíveis não impede a exibição das notícias dos demais.

12 Persistência Local

12.1 Chaves de armazenamento

CHAVE	MECANISMO	TIPO	CONTEÚDO
auth_token	SecureStorage	String	Token JWT de acesso.
user_data	SecureStorage	String (JSON)	Dados do usuário serializados em <code>snake_case</code> .
is_dark_theme	SharedPreferences	bool	Preferência de tema; padrão <code>true</code> .
locale_code	SharedPreferences	String	Código de idioma; padrão <code>'pt'</code> .
watchlist_order	SharedPreferences	List<String>	Ordem manual dos tickers.

12.2 Serialização do usuário

`UserStorageService` grava o JSON já em `snake_case`, o mesmo formato retornado pela API. A leitura pode assim reaproveitar `UserModel.fromJson` sem transformação intermediária, e um payload gravado por versão anterior permanece compatível.

A leitura é defensiva: uma falha de *parse* retorna `null` em vez de propagar exceção.

```
Future<UserModel?> readUser() async {
  final raw = await _storage.read(key: _userKey);
  if (raw == null) return null;
  try {
    return UserModel.fromJson(jsonDecode(raw) as Map<String, dynamic>);
  } catch (_) {
    return null; // dado corrompido ou de schema antigo → trata como ausente
  }
}
```

O efeito prático é que uma mudança de *schema* entre versões do aplicativo degrada para “usuário não encontrado localmente” em vez de travar a tela de perfil.

12.3 Reconciliação da ordem da watchlist

A ordem persistida é uma lista de tickers, não de objetos completos. Em `_applyOrder`, os ativos vindos do servidor são indexados por ticker e reordenados conforme a lista salva; ativos ausentes da ordem — adicionados em outro dispositivo, por exemplo — são anexados ao final:

```
final map = {for (final a in assets) a.ticker: a};
final ordered = order.where(map.containsKey).map((t) => map[t]!).toList();
final remaining = assets.where((a) => !order.contains(a.ticker)).toList();
return [...ordered, ...remaining];
```

O algoritmo é tolerante à divergência em ambas as direções: tickers salvos que já não existem são descartados, e tickers novos aparecem sem exigir invalidação da preferência.

Ausência de cache de dados de mercado

Cotações, históricos e notícias não são persistidos; toda abertura da tela inicial busca dados frescos. Para dados de mercado essa é a escolha correta — informação de preço desatualizada é pior do que ausência de informação. O único conteúdo remoto cacheado são os ícones de ativos, via `cached_network_image`, por serem imutáveis na prática.

13 Design System e Temas

13.1 Paleta

TOKEN	VALOR	APLICAÇÃO
gain / primary	#00C805	Alta de ativo, cor de marca, ações primárias.
primaryDark	#00A004	Primária no tema claro, para garantir contraste sobre fundo branco.
loss	#FF3B30	Queda de ativo, estados de erro, ações destrutivas.
bgDark	#121212	Fundo do tema escuro.
surfaceDark	#1E1E1E	Superfícies elevadas no tema escuro.
cardDark	#252525	Cartões no tema escuro.
bgLight	#F5F5F5	Fundo do tema claro.
textSecondary	#9E9E9E	Texto de apoio em ambos os temas.
divider	#2A2A2A	Separadores e bordas sutis no tema escuro.

A escolha do verde como cor primária e como cor de valorização é intencional: em um produto financeiro, a cor de marca coincidir com o sinal de ganho reforça a leitura imediata dos cartões de ativo. O par verde/vermelho segue a convenção do domínio; a direção é sempre acompanhada de ícone de seta, de modo que a informação não depende exclusivamente de cor.

13.2 Tokens de forma e componente

Ambos os temas são construídos com `useMaterial3: true` e definem explicitamente `cardTheme`, `appBarTheme`, `inputDecorationTheme`, `elevatedButtonTheme`, `chipTheme`, `dividerTheme` e `snackBarTheme`. A padronização no tema — e não em cada widget — mantém a consistência sem repetição de estilo nas telas.

TOKEN	VALOR	APLICAÇÃO
Raio de cartão	16 px	Cartões padrão do tema.
Raio de contêiner destacado	20 px	Cartão de gráfico histórico.
Raio de campo e botão	12 px	Entradas de texto e botões elevados.
Altura mínima de botão	52 px	Alvo de toque confortável em ações primárias.
Espaçamento horizontal	16 px	Margem lateral padrão das telas.

13.3 Diferenciação entre temas

Os temas não são variações mecânicas de uma mesma definição. A elevação é tratada de forma distinta em cada um: no tema escuro os cartões têm `elevation: 0` e a hierarquia é expressa por variação de

luminosidade da superfície e bordas de 0,5 px; no tema claro há elevação real e sombra suave. Essa distinção evita o resultado comum de sombras invisíveis sobre fundo escuro ou de superfícies indistintas sobre fundo claro.

O tema escuro é o padrão da aplicação, apresentado na interface como “Ativo (OLED)” — coerente com o fundo `#121212`, próximo ao preto, que reduz consumo em telas OLED.

13.4 Splash e identidade

`SplashView` combina `FadeTransition` de 800 ms com verificação de sessão de 1.200 ms, garantindo que a animação complete antes do redirecionamento. A checagem de `mounted` é feita antes e depois do `await`, prevenindo navegação sobre um widget já descartado.

14 Internacionalização

14.1 Configuração

```
# l10n.yaml
arb-dir: lib/l10n
template-arb-file: intl_en.arb
output-localization-file: app_localizations.dart
```

O projeto utiliza o pipeline oficial `flutter gen-l10n`, habilitado por `generate: true` em `pubspec.yaml`. São dois locais — `pt` (padrão) e `en` — com **38 chaves** traduzidas. O arquivo-modelo é o inglês, convenção que facilita a inclusão de novos idiomas a partir de uma base neutra.

14.2 Delegates e seleção de idioma

`MaterialApp.router` registra `GlobalMaterialLocalizations`, `GlobalWidgetsLocalizations` e `GlobalCupertinoLocalizations`, o que traduz também os componentes nativos do framework — rótulos de diálogo, seletor de data e menus de contexto.

A seleção é explícita e persistida: `LocaleNotifier` lê `locale_code` de `SharedPreferences` na construção e grava a cada troca. O idioma não segue o locale do dispositivo por padrão, decisão que privilegia previsibilidade — a escolha do usuário prevalece entre sessões.

14.3 Cobertura e limitação atual

Internacionalização parcial

A infraestrutura de i18n está completa e funcional, mas parte das *strings* permanece embutida em português diretamente nos widgets — rótulos da barra de navegação ("Início", "Perfil", "IA Chat", "Config"), títulos de seção da Home ("Watchlist", "Gráficos", "Notícias"), mensagens de erro dos *view models* e as respostas do assistente. Com o idioma em EN-US, essas partes seguem exibindo texto em português. A correção é mecânica — mover cada literal para os arquivos `.arb` e substituir por `AppLocalizations.of(context)` — mas ainda não foi concluída.

As mensagens de erro representam um caso à parte: por estarem nos *view models*, que não possuem `BuildContext`, a externalização exige uma decisão adicional de projeto — retornar chaves de erro em vez de texto pronto, deixando a tradução para a camada de view.

15 Estratégia de Testes

A suíte contém **93 casos de teste** distribuídos em 10 arquivos, com a árvore de testes espelhando a estrutura de `lib/`.

15.1 Distribuição

ARQUIVO	CASOS	COBERTURA
<code>core/utils/validators_test.dart</code>	20	Formato de e-mail, comprimento de senha, campos obrigatórios e casos de borda.
<code>core/constants/api_endpoints_test.dart</code>	13	Construção de cada rota, incluindo interpolação de <code>ticker</code> e <code>id</code> .
<code>core/utils/formatters_test.dart</code>	11	Moeda, data, tempo relativo e rótulo de direção; entradas malformadas.
<code>presentation/viewmodels/profile_viewmodel_test.dart</code>	10	Busca com <i>debounce</i> , seleção, sincronização diferencial e atualização de perfil.
<code>presentation/viewmodels/auth_viewmodel_test.dart</code>	8	Login, cadastro com auto-login, logout e mapeamento de erros.
<code>data/repositories/mock_chat_repository_test.dart</code>	7	Correspondência por palavra-chave e resposta genérica.
<code>presentation/viewmodels/chat_viewmodel_test.dart</code>	7	Envio, indicador de digitação, entrada vazia e degradação em erro.
<code>core/constants/api_constants_test.dart</code>	6	Base URL e integridade do mapa de períodos.
<code>presentation/viewmodels/home_viewmodel_test.dart</code>	6	Carga, watchlist vazia, troca de período, reordenação e erro.
<code>presentation/views/login_view_test.dart</code>	5	Renderização, validação de formulário e interação de submissão.

15.2 Abordagem

- **Dublês sobre abstrações do domínio.** `test/helpers/fake_repositories.dart` concentra os *mocks* — `MockAuthRepository`, `MockAssetRepository`, `MockWatchlistRepository`, `MockChatRepository` e `MockFlutterSecureStorage`. Todos implementam as interfaces do domínio, não as classes concretas, o que valida o contrato e mantém os testes indiferentes a mudanças de implementação.
- **Mocktail sem geração de código.** Dispensa passo de `build_runner` para atualizar dublês, reduzindo atrito ao alterar assinaturas.

- **Isolamento por `ProviderContainer`**. Cada teste monta seu próprio contêiner com sobrescritas e o descarta em `addTearDown`, garantindo ausência de estado compartilhado entre casos.
- **Prioridade à lógica pura e à orquestração.** A cobertura concentra-se em validadores, formatadores, construção de rotas e *view models* — as áreas onde defeitos são silenciosos e a verificação automatizada tem melhor retorno.

15.3 Lacunas

ÁREA	SITUAÇÃO	CONSIDERAÇÃO
Testes de widget	1 DE 7 TELAS	Apenas <code>LoginView</code> é coberta. <code>test/widget_test.dart</code> está deliberadamente vazio, com comentário indicando que testes de UI serão adicionados quando os componentes estabilizarem.
Testes de integração	AUSENTE	Não há diretório <code>integration_test/</code> ; os fluxos ponta a ponta — cadastro → login → watchlist → gráficos — não têm verificação automatizada.
Repositórios HTTP	AUSENTE	As implementações que usam Dio não são testadas contra um adaptador simulado; a desserialização de respostas reais não é verificada.
Relatório de cobertura	NÃO CONFIGURADO	<code>flutter test --coverage</code> é executável, mas não há limiar definido nem publicação do relatório.

15.4 Execução

```
flutter test           # suíte completa
flutter test --coverage # gera coverage/lcov.info
flutter test test/core/ # subconjunto por diretório
flutter analyze        # análise estática
```

16 Registro de Decisões Técnicas

Consolidação das decisões estruturais, com alternativas consideradas e o custo assumido em cada caso.

#	DECISÃO	MOTIVAÇÃO	ALTERNATIVA PRETERIDA E CUSTO ASSUMIDO
01	Clean Architecture em 3 camadas	Isola a UI dos detalhes de transporte e permite substituir implementações sem tocar na apresentação — condição para testar <i>view models</i> sem rede.	Arquitetura em camada única. Rejeitada: acoplaria as telas ao Dio e inviabilizaria os testes de <i>view model</i> .
02	Modelos de dados compartilhados com o domínio	O modelo de leitura do cliente espelha o contrato da API; entidades separadas com mapeadores dobrariam o código sem isolar comportamento.	Entidades de domínio + <i>mappers</i> . Custo assumido: mudanças de <i>schema</i> no backend propagam até a apresentação.
03	Riverpod como DI e estado	Resolução segura em tempo de compilação, sobrescrita trivial em testes, sem <i>singleton</i> global.	Provider + GetIt. Rejeitada: dois mecanismos para um problema, com registro global mutável.
04	<code>StateNotifier</code> em vez de <code>AsyncNotifier</code>	Estado explícito com múltiplos sinalizadores (<code>loading</code> , <code>refreshing</code> , <code>newsLoading</code>) que <code>AsyncValue</code> não modela naturalmente.	API moderna do Riverpod. Custo assumido: dívida frente à direção do ecossistema; migração mecânica pendente.
05	Freezed + json_serializable	Imutabilidade, igualdade estrutural e serialização sem código manual sujeito a divergência silenciosa.	Classes escritas à mão. Rejeitada: <code>copyWith</code> e <code>==</code> manuais são fonte recorrente de defeito.
06	Dio com interceptador	Injeção de token e tratamento de 401 em ponto único; nenhum repositório conhece autenticação.	<code>package:http</code> . Rejeitada: exigiria repetir cabeçalhos e verificação de status em cada chamada.
07	Guarda de rota assíncrona	Decisão sempre baseada no estado real da credencial, inclusive após limpeza pelo interceptador.	Sessão em memória com <code>refreshListenable</code> . Custo assumido: uma leitura de I/O por navegação.
08	<code>StatefulShellRoute.indexedStack</code>	Preserva pilha e rolagem por aba, com URL sincronizada, sem gerência manual de índice.	<code>IndexedStack</code> sob rota única. Rejeitada: exigiria sincronizar estado e URL manualmente.
09	Base URL por <code>dart-define</code>	Um único artefato de código serve todos os ambientes; nenhum segredo de ambiente versionado.	Arquivo <code>.env</code> . Rejeitada: adiciona dependência e um arquivo a mais no fluxo de build.
10	Separação SecureStorage / SharedPreferences	Dados sensíveis sob cifra do sistema; preferências em armazenamento simples,	Tudo em armazenamento seguro. Rejeitada:

#	DECISÃO	MOTIVAÇÃO	ALTERNATIVA PRETERIDA E CUSTO ASSUMIDO
		com leitura síncrona após inicialização.	sobrecarga desnecessária para preferência de tema.
11	Carga paralela na Home	<code>Future.wait</code> faz a latência tender ao maior tempo individual, e não à soma das chamadas.	Requisições sequenciais. Rejeitada: tempo de carga cresceria linearmente com o tamanho da watchlist.
12	Notícias fora do caminho crítico	Cotações e gráficos renderizam sem esperar o <i>endpoint</i> mais lento e menos essencial.	Carga única aguardando tudo. Rejeitada: a informação principal ficaria refém da secundária.
13	Sincronização diferencial da watchlist	No máximo duas requisições ao salvar, independentemente do número de alterações.	Uma requisição por toque. Rejeitada: tráfego excessivo e estado inconsistente em falha parcial.
14	Repositório de chat simulado	Permite desenvolver e validar a UI de conversa antes da existência do serviço de IA, atrás do contrato definitivo.	Adiar a funcionalidade. Rejeitada: a UI é substancial e não deveria bloquear em dependência externa.
15	Erros por exceção, sem <code>Result</code>	Menos cerimônia; cada <i>view model</i> traduz a falha no seu <code>catch</code> para uma mensagem de usuário.	<code>Either / Result</code> . Custo assumido: as assinaturas não declaram as falhas possíveis.
16	Provider colocado junto da implementação	Definição próxima do uso; sem arquivo central que acople todos os módulos.	Arquivo único de injeção. Rejeitada: ponto de acoplamento global e conflitos frequentes de merge.

17 Build, Configuração e Distribuição

17.1 Preparação do ambiente

```
flutter pub get
dart run build_runner build --delete-conflicting-outputs # freezed + json_serializable
flutter gen-l10n # localizações
```

A etapa de `build_runner` é **obrigatória** após clonar o repositório ou alterar qualquer modelo: os arquivos `*.freezed.dart` e `*.g.dart` são gerados e não devem ser editados à mão.

17.2 Configuração de ambiente

A única variável de ambiente é a origem da API, lida em tempo de compilação:

```
# desenvolvimento (usa o padrão http://localhost:8000)
flutter run

# ambiente remoto
flutter run --dart-define=API_BASE=https://api.exemplo.com

# release Android
flutter build apk --release --dart-define=API_BASE=https://api.exemplo.com

# release iOS
flutter build ipa --release --dart-define=API_BASE=https://api.exemplo.com
```

Por ser `String.fromEnvironment` com `const`, o valor é resolvido na compilação e sujeito a *tree shaking* — não há custo de leitura em tempo de execução, e o valor não pode ser alterado sem novo build.

17.3 Alvos de compilação

PLATAFORMA	CONFIGURAÇÃO	OBSERVAÇÃO
Android	<code>com.example.easy_finance</code>	<code>compileSdk</code> , <code>minSdk</code> e <code>targetSdk</code> herdados dos padrões do Flutter. O <i>application id</i> precisa ser alterado antes da publicação.
iOS	Deployment target 13.0	<code>Runner.xcworkspace</code> configurado; requer assinatura e perfil de provisionamento.
Web	<code>web/index.html</code>	Manifesto e ícones presentes; layout não otimizado para telas largas.
Windows / macOS / Linux	CMake / Xcode	<i>Runners</i> gerados e funcionais; sem ajuste específico de UI.

17.4 Recomendações para o pipeline de release

- Fixar `API_BASE` em origem **HTTPS** na configuração do pipeline, jamais dependendo do *fallback* local.

2. Habilitar ofuscação: `--obfuscate --split-debug-info=build/symbols` , arquivando os símbolos para desofuscação de *stack traces*.
3. Alterar o *application id* Android de `com.example.*` para o domínio da organização.
4. Executar `flutter analyze` e `flutter test` como etapas bloqueantes antes do empacotamento.
5. Versionar o incremento de `version:` em `pubspec.yaml` junto ao artefato gerado.

18 Limitações Conhecidas e Roadmap

As limitações abaixo são conhecidas e deliberadas no estado atual do produto. Estão listadas com o impacto correspondente para permitir priorização informada.

18.1 Limitações

LIMITAÇÃO	IMPACTO	DETALHE E CAMINHO DE RESOLUÇÃO
Assistente sem IA real	FUNCIONAL	Respostas provêm de dicionário local por palavra-chave. Resolução: nova implementação de <code>ChatRepository</code> e troca de uma linha em <code>chatRepositoryProvider</code> .
i18n parcial	UX	Rótulos de navegação, títulos de seção e mensagens de erro embutidos em português. Requer externalização para <code>.arb</code> e decisão sobre chaves de erro nos <i>view models</i> .
Sem <i>refresh token</i>	UX	Sessão expirada exige reautenticação manual. Depende de suporte no backend.
Sem certificate pinning	SEGURANÇA	Bloqueante para distribuição pública. Implementável via <code>badCertificateCallback</code> do Dio.
Cobertura de widget baixa	QUALIDADE	Seis das sete telas sem teste de widget; nenhum teste de integração.
Endpoints ociosos	ESCOPO	<code>getDividends</code> , <code>getFinancials</code> e exclusão de conta implementados, sem consumidor.
Sem paginação na busca	ESCALA	<code>GET /assets?search=</code> retorna a lista completa; consultas amplas podem gerar resposta volumosa.
Ausência de observabilidade	OPERAÇÃO	Sem relatório de <i>crash</i> ou telemetria; falhas em produção não são visíveis. Recomenda-se Crashlytics ou Sentry.
Layout não responsivo	PLATAFORMA	Dimensões calibradas para telas de telefone; alvos web e desktop compilam mas não aproveitam a largura disponível.
<i>Application id</i> de exemplo	PUBLICAÇÃO	<code>com.example.easy_finance</code> impede publicação nas lojas. Alteração obrigatória.

18.2 Evolução sugerida

CURTO PRAZO — HABILITAR PUBLICAÇÃO

1. Alterar o *application id* Android e configurar assinatura de release em ambas as plataformas.
2. Fixar `API_BASE` HTTPS e habilitar ofuscação no pipeline.
3. Concluir a externalização de *strings* para os arquivos `.arb`.
4. Integrar relatório de *crash*.

MÉDIO PRAZO — COMPLETAR O PRODUTO

1. Construir a tela de detalhe do ativo, consumindo `getFinancials` e `getDividends` já disponíveis.
2. Substituir `MockChatRepository` por integração com serviço de IA, mantendo o contrato existente.
3. Ampliar a cobertura de testes de widget e introduzir `integration_test/` para o fluxo principal.
4. Implementar exclusão de conta, consumindo o `endpoint` já mapeado.

LONGO PRAZO — ESCALA E RESILIÊNCIA

1. Migrar de `StateNotifier` para `AsyncNotifier`, aproveitando `riverpod_generator`.
2. Introduzir paginação na busca de ativos.
3. Avaliar atualização de cotações em tempo real por WebSocket, substituindo o modelo de *pull*.
4. Adotar layout adaptativo para tablet, web e desktop.
5. Implementar *certificate pinning* e, quando disponível no backend, rotação de token.

Finance Pro — Especificação Técnica · versão 1.0.0+1 · 13/08/2026

Documento derivado da análise direta do código-fonte do repositório `easy_finance` (branch `main`). Situações marcadas como ausentes ou parciais refletem o estado verificado do código na data de emissão.